

Bandwidth-Efficient Qwen3.8 Decoding on AMD RDNA4 with a Verified Integer Core and Multi-Token Prediction

Author information pending

Research draft — October 2026

Abstract

Running a large hybrid language model on one graphics processor requires efficient weight streaming, recurrent-state management, and careful numerical implementation. This paper describes `amd-infer`, a C inference engine specialized for the Qwen3.8-27B Q8_0 model recorded in the project repository and the AMD RDNA4 gfx1201 architecture. The engine combines row-major quantized weight packing, wave-level integer dot products, fused hybrid-attention operations, streaming model loading, and multi token prediction (MTP). A shared core has accompanying Lean proofs for indexing, quantized integer dot products, reduction identities, and activation-plan checking. This verification boundary excludes floating-point attention, compiler correctness, device synchronization, and speculative decoding. Repository records report 21.7 tokens/s for ordinary decoding on a Radeon AI PRO R9700, corresponding to approximately 590 GB/s of effective weight bandwidth. With three MTP drafts, recorded throughput reaches 60.1 tokens/s on a coding prompt and 50.6 tokens/s on a text-continuation prompt. The records also report greedy-token agreement with ordinary decoding and a WikiText-2 perplexity of 5.1804 on four short evaluation chunks. These measurements were not rerun for this manuscript, and raw benchmark logs are unavailable in the inspected snapshot. The design illustrates how a narrow verified core can coexist with device-specific optimization while keeping empirical performance and numerical claims separate from proof-backed properties.

Keywords: quantized inference; weight reuse; recurrent state; speculative decoding; memory planning; formal verification

1 Introduction

Single-sequence autoregressive decoding repeatedly applies large weight matrices to a small number of activation vectors. For a quantized model that occupies most of a device’s memory, repeatedly reading those matrices makes weight traffic a central engineering constraint. Hybrid attention adds another requirement: speculative execution must recover both position-indexed attention caches and recurrent state after rejecting a draft token.

Multi token prediction (MTP) is the speculation mechanism examined here. This paper studies the implementation in the `amd-infer` repository, focusing on the model identified there as Qwen3.8-27B Q8_0 and stored under the GGUF architecture identifier `qwen35`. The name, layer configuration, and measurements are repository observations, rather than independently established model-release claims. The inspected snapshot is commit `ff3c7f3f465bdfce9a9a2f0050fbac6db6af82fc`. Its README describes a 64-layer model containing 48 linear-attention layers and 16 full-attention layers, with a 28.6 GB model file. The paper makes no claim that this implementation is optimal across devices or inference frameworks.

The system offers three concrete engineering contributions. First, a compact C engine exposes weight layout and wave-level work assignment directly, using eight-bit integer dot products for Q8_0 matrices. Second, hybrid-attention execution combines a fused recurrent kernel with state snapshots for bounded speculation. Third, a shared integer core connects implementation functions to accompanying Lean proofs, including a checker for activation memory plans. The proof boundary is deliberately explicit: the full decoder is not formally verified.

The evaluation describes existing project records. It addresses how close ordinary decoding comes to the project’s stated memory-bandwidth ceiling and how multi-token prediction changes useful token throughput. It also documents numerical checks and the limitations of the available evidence.

2 Background and Related Work

Quantized language-model inference separates integer representations from floating-point scaling. The repository’s Q8_0 format uses a half-precision scale and 32 signed eight-bit values per block. Activations use their own block scale. Integer products can therefore be accumulated before converting the result to floating point and applying the two scales.

Speculative sampling evaluates draft continuations with a target model and applies an acceptance rule to preserve a target distribution [1]. The engine examined here implements a narrower greedy variant. Drafts are accepted only when they match the target model’s argmax token. No distribution-preserving stochastic sampler is implemented or claimed. The use of the model’s multi-token prediction head also differs from deploying an unrelated small draft model.

WikiText was introduced with the pointer-sentinel language-model study [2]. The recorded quality check uses only four WikiText-2 chunks of 512 tokens each, so it is a short diagnostic rather than a full-corpus or task-level evaluation. References in this paper provide conceptual context; implementation and performance claims are grounded in the repository files listed in Appendix A.

3 Engine Architecture

3.1 Loading and execution graph

The host parses GGUF (GGML Universal File Format) version 3 metadata and tensors, builds an operator list, and assigns activation buffers to a shared arena. Central processing unit (CPU) and graphics processing unit (GPU) backends consume the same graph and offsets. The GPU host dynamically loads the Heterogeneous-compute Interface for Portability (HIP) runtime, while the kernels are compiled as C for `amdgcncn-amd-amdhsa`, targeting `gfx1201`. Runtime HIP availability remains necessary even though kernel compilation does not require a conventional HIP C++ source pipeline.

Projection matrices are concatenated when their storage types permit it. In a linear-attention layer, the query, key, value, output-gate, beta, and alpha projections are combined into one matrix operation. Gate and up projections in the feed-forward path are likewise combined. Bias, residual addition, and production of quantized activation copies are fused into selected kernels. These choices reduce intermediate launches and avoid separate quantization passes where supported.

GPU loading is streaming: each repacked matrix is copied to device memory and its temporary host storage is released. The project records 516 uploaded arrays totaling 28.57 GB and approximately 50 seconds of loading time for the 27B model. The Q8_0 embedding remains quantized and is gathered by row, avoiding expansion of the full embedding matrix to four-byte floating-point values. These quantities are reported observations, not newly measured memory bounds.

3.2 Activation memory planning

The planner sorts buffers by size and chooses the lowest available 256-byte-aligned offset that does not conflict with an already placed live buffer. Lifetimes are closed intervals of operator indices. If buffers i and j are simultaneously live, their byte intervals must be disjoint:

$$[o_i, o_i + s_i) \cap [o_j, o_j + s_j) = \emptyset. \tag{1}$$

The candidate plan is checked by `ie_plan_check`; execution aborts if it is rejected. Accompanying proofs establish arena containment, alignment, and non-overlap for accepted plans under the formal model. The greedy construction algorithm itself is not proved optimal or correct. Actual allocation lengths, faithful lifetime annotations, and the caller’s use of offsets remain obligations outside the checker theorem.

4 Quantized Kernels and the Verified Core

4.1 Q8 layout and dot products

A Q8_0 matrix is repacked into separate row-major arrays of integer words and half-precision scales. Each 32-value block occupies eight 32-bit words in the integer array. For row r , block b , word w , and n_b blocks per row, the destination word index is

$$i(r, b, w) = 8(rn_b + b) + w, \quad 0 \leq w < 8. \quad (2)$$

The accompanying address and inverse-map laws connect this layout to the source bytes. The current Q8 size predicate requires $r_{\text{count}} < 2^{18}$, $n_b < 2^{11}$, and $r_{\text{count}}n_b < 2^{26}$. These are the current code limits; an earlier README paragraph contains a smaller, superseded Q8 bound.

For block b , weights are represented by scale s_b and signed integers q_{bj} . Activations use a_{bj} and a floating-point scale d_b , computed from the maximum absolute activation value divided by 127. The output approximation is

$$y_r = \sum_{b=0}^{n_b-1} s_{rb} d_b \left(\sum_{j=0}^{31} q_{rbj} a_{bj} \right). \quad (3)$$

Eight packed dot4 operations implement the inner integer dot product. The core models signed-byte interpretation and accumulation using 32-bit unsigned arithmetic. Integer identities are proved modulo 2^{32} ; they do not establish exact real-valued output equivalence after quantization and floating-point scaling.

4.2 Wave assignment and memory access

Each workgroup contains 256 threads, organized as eight 32-lane waves. Wave v in workgroup g owns row $8g + v$. Lane ℓ handles blocks $\ell + 32t$. The nominal grid has $\lceil R/8 \rceil$ workgroups for R rows. Coverage and uniqueness laws establish the intended assignment within the modeled domain.

The kernels stage two blocks of loads before performing their arithmetic. Weight accesses use 128-bit vector loads, and wave-level reductions use data-permutation operations rather than shared-memory permutation. The chosen reduction order is retained across CPU and GPU implementations. This ordering limits one source of numerical discrepancy but cannot make floating-point addition associative.

The GPU launcher pads nominal grids of 253–256 workgroups to 257. The README records 17.0 microseconds at 256 groups and 6.5 microseconds at 257 for a 2048-row, 48-block microbenchmark on the R9700. Additional groups exit without producing rows. The cause of this discontinuity is unknown, and the policy should be treated as a device-specific observation rather than a general scheduling rule.

4.3 Verification boundary

The C and C++ interfaces include the same `core/ie_core.h` source. The project translates its restricted functional C++ view to Lean using `cpp-lean` and the Edison Design Group (EDG) front end. Proof files cover work assignment, Q4 and Q8 addresses and packing, integer dot products, integer reduction identities, and plan-checker soundness. Q4 proof support exists in the shared core but is not the weight path evaluated in this paper.

The trusted boundary includes the translation from source semantics to Lean, host and GPU compiler correctness, the dot4 instruction matching its modeled behavior, valid caller-supplied lengths and size predicates, and the device memory model. Floating-point normalization, rotary embeddings, softmax, gating, cross-workgroup synchronization, parsing, and speculative decoding are outside these proofs. The proof sources were inspected for this manuscript; the Lean regeneration and checking command was not rerun. Consequently, this paper reports the provided formal artifacts and their stated scope without claiming a fresh proof-validation receipt.

5 Hybrid Attention and Multi-Token Prediction

5.1 Fused recurrent attention

The recorded architecture places three linear-attention layers before each full-attention layer. A linear layer uses a gated delta recurrence with a 128×128 state matrix per value head. Its fused kernel applies a four-tap causal convolution and SiLU, normalizes query and key vectors in the Euclidean norm, computes beta and decay gates, updates state, and applies gated root mean square normalization (RMSNorm) to the output.

Let S_{p-1} be the previous key-by-value state, and let q_p, k_p, v_p denote the processed query, key, and value. With $\beta_p = \text{sigmoid}(b_p)$ and $\lambda_p = \exp(\text{softplus}(a_p + \delta_b)A)$, the implemented recurrence is

$$\tilde{S}_p = \lambda_p S_{p-1}, \quad (4)$$

$$\Delta_p = \beta_p (v_p - \tilde{S}_p^\top k_p), \quad (5)$$

$$S_p = \tilde{S}_p + k_p \Delta_p^\top, \quad (6)$$

$$o_p = S_p^\top q_p / \sqrt{128}. \quad (7)$$

A value head selects key head $h \bmod n_k$. The output is normalized, multiplied by a learned weight and SiLU(z_p), and optionally quantized for the next projection. At position zero, the kernel treats prior state as zero without reading it.

A workgroup owns one value head. State loads are issued early to overlap with convolution computation. The README reports approximately 6.9 microseconds per recurrent-kernel invocation for the smaller Qwen3.5-0.8B workload; this is supporting implementation context and is not a measured 27B kernel latency.

5.2 Full attention

Full-attention layers allocate position-indexed key-value (KV) caches. Queries and keys receive per-head RMSNorm, and the rotary position transformation operates on the first 64 dimensions with NEOX pairing. Query projections also supply a gate; the attention output is multiplied by its sigmoid.

Attention is split across query heads and 32-position segments. Waves form online-softmax summaries, a workgroup combines local summaries, and the final completing workgroup combines segment maxima, denominators, and weighted values. These operations use shared memory and atomic completion counters. Their synchronization is tested implementation behavior, rather than a theorem of the integer core.

5.3 Bounded speculation and weight reuse

The engine accepts one to three MTP drafts through `--draft D`. The prediction head first catches up previously accepted positions using the target model’s hidden states. It predicts the first draft from a true hidden state and the current token; later drafts use its own predicted hidden state. The target then evaluates the current token and all D drafts in one pass.

For Q8 projections, `ie_gemv_q8q8_t` maintains up to four activation accumulators while loading each weight block once. For each activation vector it preserves the single-token arithmetic order. Other operations are executed in token order where necessary; this is limited verification batching, not a general dense matrix-multiplication engine.

If target predictions are $a_0, \dots, a_D$ and drafts are $d_1, \dots, d_D$, acceptance chooses the longest prefix satisfying $d_j = a_{j-1}$, then emits the target prediction a_k after k accepted drafts. This yields ordinary greedy tokens provided the batched target computation and restored state match ordinary execution. The README reports such token agreement for tested draft depths, but it is not a formal end-to-end equivalence proof.

Recurrent state uses four slots. Within a verification step starting from slot c , state after token index t is written to $(c + t) \bmod 4$. After accepting k drafts the host retains slot $(c + k) \bmod 4$. The convolution input ring has eight slots, preserving the history needed for at most four verified tokens and a four-tap convolution. KV entries are overwritten by position on subsequent execution. This storage protocol makes rollback possible without copying every recurrent matrix after rejection.

6 Recorded Evaluation

6.1 Evidence and protocol

All numerical results below are transcribed or calculated from the repository README at the inspected commit. No GPU benchmarks, model downloads, or full-corpus evaluations were conducted for this manuscript. Raw per-run timing logs, exact benchmark prompt strings, model-file hashes, and a complete machine configuration are absent. The recorded device is a Radeon AI PRO R9700, targeting gfx1201; the README identifies a ROCm 7.1/7.14 runtime environment and states a nominal memory bandwidth of 640 GB/s without a more precise runtime-build record.

6.2 Ordinary decoding and bandwidth

Table 1 reports ordinary Qwen3.8-27B decoding. The README states that the engine ran each length three times with less than 0.1% variation, but provides no individual samples or definition of that variation. Effective bandwidth counts approximately 27.2 GB of weights per generated token. It is a workload-derived ratio, not a hardware-counter measurement of all memory traffic. Using the 64-token latency,

Table 1: Recorded ordinary Qwen3.8-27B Q8_0 decoding on R9700.

Generated tokens	Milliseconds/token	Tokens/s
64	46.01	21.7
512	46.17	21.7

$$B_{\text{eff}} = \frac{27.2 \text{ GB}}{0.04601 \text{ s}} \approx 591 \text{ GB/s}, \quad \frac{B_{\text{eff}}}{640 \text{ GB/s}} \approx 92.4\%. \quad (8)$$

The source rounds effective bandwidth to 590 GB/s and utilization to 92%. Separate Q8 shape microbenchmarks for the 27B projections report 589–634 GB/s. Their proximity to the stated bandwidth ceiling is consistent with weight traffic dominating these operations, but does not establish a counter-based bottleneck diagnosis for every kernel.

6.3 MTP throughput

Table 2 reports 300-token generation runs. The coding and continuation prompts are named by category in the README, without exact prompt bytes. Acceptance is reported as accepted drafts divided by attempted drafts, consistent with the host’s `steps`× D denominator. Speedups in the table are calculated against the recorded 21.7 tokens/s ordinary rate. The chat ratio uses the same ordinary rate; a separate matched chat

Table 2: Recorded Qwen3.8-27B MTP results; speedups are arithmetic ratios.

Prompt category	Drafts	Tokens/s	Acceptance	Speedup
Coding	0	21.7	—	1.00
Coding	1	38.4	98.7%	1.77
Coding	2	51.3	97.1%	2.36
Coding	3	60.1	93.2%	2.77
Text continuation	0	21.7	—	1.00
Text continuation	1	36.0	86.9%	1.66
Text continuation	2	45.6	81.6%	2.10
Text continuation	3	50.6	74.2%	2.33
Chat	3	47.5	67.3%	2.19

baseline is not reported. Acceptance falls as draft depth increases, yet measured throughput rises through

three drafts in the two reported depth sweeps. This behavior is consistent with amortizing the large target weight read over more useful tokens. The data do not justify extrapolation to other prompts or larger draft depths.

As a simple accounting model, if K drafts are accepted, a step produces approximately $1 + K$ useful tokens and costs target verification plus draft work. Thus

$$\text{throughput} \approx \frac{1 + \mathbb{E}[K]}{t_{\text{verify}} + t_{\text{draft}}}. \quad (9)$$

The README estimates roughly 63 ms per step at three drafts, compared with about 46 ms per ordinary token, and attributes additional work mainly to the MTP and vocabulary-output projections. This attribution is a source hypothesis, not a measured decomposition. Each draft reads approximately 0.45 GB of prediction-head weights and 1.35 GB of output-layer weights according to the records.

6.4 Numerical checks

The recorded Qwen3.8-27B perplexity is 5.1804 on four WikiText-2 chunks of 512 tokens. Only the latter 255 predicted tokens of each chunk are scored. If their negative log-probabilities are ℓ_i , perplexity is $\exp(\sum_i \ell_i / N)$, giving $N = 1020$ scored predictions across the four chunks. This small sample does not establish full-corpus perplexity or downstream task quality.

The README reports identical greedy tokens between speculative and ordinary engine execution at draft depths one through three for tested 0.8B and 27B cases. It also records a plausible response to a short sky-color prompt. The latter is a smoke check rather than a task benchmark. CPU dot-product tests and small synthetic-model comparisons exercise shared integer and floating-point paths, but do not independently validate the entire 27B recurrent computation. No exact-token transcript or per-position 27B logit dump is included in the snapshot.

7 Limitations and Reproducibility

The available evidence supports an implementation account and a bounded report of prior measurements. It cannot support statistical significance, broad prompt-level generalization, or superiority across inference frameworks. No cross-framework comparison is included in this paper. Backend versions, power settings, clocks, warm-up behavior, exact prompts, and run-level timing samples must be recorded before making stronger empirical claims.

The model name Qwen3.8 and the external MTP weight provenance are carried from the README. The records describe extracting approximately 451 MB of MTP tensors from another Q8_0 artifact. Full artifact identifiers, hashes, licensing, and compatibility metadata should be retained for a reproducible release; independent model documentation was not verified here. Model naming should therefore not be interpreted as a verified release history.

Proof-backed properties concern a restricted integer core. The formal tools, compiler chain, memory allocation, runtime state transitions, and floating-point arithmetic remain trusted or empirically checked. Particularly consequential unproved mechanisms include atomic completion in segmented attention and restoring recurrent state after speculative rejection. The old README’s blanket untested-model statement conflicts with its later 27B results; this paper uses the later model-specific records and does not infer complete model or context coverage.

A reproduction should first regenerate and check Lean proofs with the documented translator and tool versions, then run integer and synthetic-model checks, record model hashes, and capture device/runtime metadata. Ordinary and speculative runs should use identical prompt tokens and stopping rules, retain output-token sequences, and log individual timings and accepted-prefix lengths. GPU profiling should be performed separately from throughput measurements because per-kernel instrumentation changes execution overhead.

8 Conclusion

amd-infer combines quantized wave-level computation, fused hybrid attention, streaming loading, checked activation planning, and bounded MTP verification for the repository’s Qwen3.8-27B workload. Recorded ordinary decoding reaches approximately 92% of the project’s stated weight-bandwidth ceiling. Three-draft MTP reaches 60.1 tokens/s on the recorded coding prompt and 50.6 tokens/s on text continuation. These observations suggest that weight reuse can increase useful decoding throughput even when ordinary execution already streams weights efficiently. The accompanying integer proofs provide a narrow foundation for layout, dot products, work assignment, and plan checking; wider correctness and performance claims require additional empirical and formal evidence.

Declarations

Data and code availability. The analyzed source and proof artifacts are in the accompanying repository at the commit given above. Results originate from its README. Source hashes and reference-verification receipts accompany this manuscript. Raw GPU measurements and model artifacts are not included in the inspected snapshot.

Ethics declaration. This manuscript describes software and recorded machine benchmarks. No human-participant study was performed in preparing it. Model and dataset licensing require confirmation by the authors before redistribution.

Author contributions. Contributor identities and Contributor Roles Taxonomy (CRedit) assignments require author confirmation. This draft does not attribute implementation or proof work to an unconfirmed person.

Conflict of interest. Author declarations have not been supplied and remain to be confirmed.

Funding. Funding information has not been supplied and remains to be confirmed.

AI assistance. A Codex assistant inspected the source and drafted this manuscript. It also prepared the LaTeX build. It did not rerun the GPU measurements or Lean proofs. The authors retain responsibility for the technical claims, source attribution, and final manuscript.

A Artifact Map and Build Instructions

Artifact	Evidence role
README.md	Reported performance, perplexity, model configuration
core/ie_core.h	Integer functions, layout and size predicates
laws/ie_laws.cpp, proofs/	Law statements and Lean proofs
src/model.c, src/plan.c	Graph construction and checked arena planning
src/hip.c	Device dispatch, streaming and multi-token execution
src/main.c	Greedy acceptance and state-slot selection
kernels/ie_kernels.c	Quantized, attention and recurrent kernels
tools/gemv_bench.c	Projection microbenchmark implementation

The document is built from the repository root with:

```
tectonic --keep-logs --keep-intermediates --outdir paper paper/main.tex
```

The engine’s documented checks are `make test` and `make proofs`. The latter requires cpp-lean, EDG, and Lean 4.34.0. They are reproduction instructions, not checks executed in preparing this manuscript.

References

- [1] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” arXiv:2302.01318, 2023. <https://doi.org/10.48550/arXiv.2302.01318>
- [2] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer Sentinel Mixture Models,” arXiv:1609.07843, 2016. <https://doi.org/10.48550/arXiv.1609.07843>